Free and Open Source Silicon

Working together for Free and Open Source Silicon

BlackParrot UVM Testbenches

Google Summer of Code

Contents

1 Contact Information **3**

2 About Project **3**
2.1 Impact and Objectives 3

3 Prior Work Done **4**
3.1 My Understanding of UVM 4
3.2 Candidate Modules Chosen 6

4 Plan **7**
4.1 Verification Methodology: From Directed to UVM 7
4.2 Risks In This Plan 7
4.3 Backup Plan 8

5 Timeline **8**

6 Deliverables **10**

7 Availability **10**

8 Schedule To Update Community **10**

9 References **10**

1 Contact Information

Full Name: Baratam Nidhishri
Primary Email: nidhibaratam2005@gmail.com
Academic Email: 23uec527@lnmiit.ac.in
LinkedIn: www.linkedin.com/in/baratamnidhishri
GitHub: https://github.com/nidhibaratam
Portfolio: https://nidhibaratam.github.io/my_portfolio/
Phone No: +91 8367047097
Address: Visakhapatnam, Andhra Pradesh, India
Timezone: IST (UTC +5:30)

2 About Project

BlackParrot aims to be the default Linux-capable host multicore for and by the world. This processor mainly manifests itself to be ***Tiny*** , ***Modular*** and ***Friendly***. The four success metrics of **BlackParrot** are ***QUALITY*** , ***VIRALITY*** , ***EFFICIENCY*** and ***FUNCTIONALITY***.

The architecture is built around a sophisticated *40-bit physical address routing hierarchy*, which separates global memory from local tile resources. I previously suggested the visualization of this routing logic for the official repository (see Figure 1), which was later adopted into tables in platform guide of the **BlackParrot**.

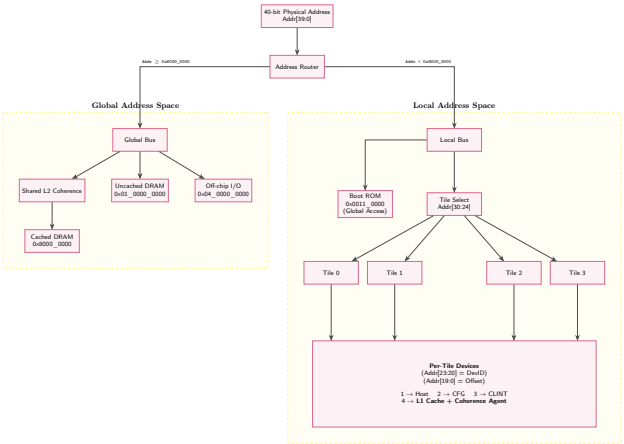

Figure 1: Detailed BlackParrot Address Routing Hierarchy

We know that **BlackParrot** is an open-source Linux-capable RISC-V multicore with a long history of being used to stress-test open-source EDA tooling. We will be using Verilator to run the tests. Verilator is used mainly because it has added UVM support.

2.1 Impact and Objectives

- Improves detection of bugs and edge cases.
- Improves Verilator based verification and bridges gaps for new processor designs.
- Creates reusable test components that work across different parts of the BlackParrot processor, saving time on future designs.

3 Prior Work Done

My environment for this project is on macOS (Apple Silicon). I had used the official workflow from black-parrot-sim repository and followed this to set up the development environment.

- I first cloned the black-parrot-sim repo
- Build the docker container file and build the image
- I then used prep-lite for the set up of black-parrot-sim repo for environment verilator and necessary toolchains.

My merged pull requests:

- PR #1252 :
  - Fixes Issue : #1251
  - Add BlackParrot bp_top and bp_me Simulation Examples to the evaluation guide.
  - bp_top simulation examples consists of **Hello World** and **RISC-V ISA Simple Test**
  - bp_me simulation examples consists of **Clean Build with Waveform Support**, **Random Coherence Stress**,**Set Hammer Test** and **Trace-Based Deterministic Test**
- PR #1278 :
  - Fixes Issue : #790 ,This is a follow-up to PR #1255.
  - Add platform address maps and routing details to the platform guide in docs.
  - I first proposed a Mermaid.js flowchart which was later upgraded to work offline too as a markdown table into the document.

3.1 My Understanding of UVM

After the above PRs I have built a UVM sample module based on a Synchronous FIFO RTL Design.

```
[1946] [SCB] MATCH: Expected 56, Got 56
[1956] [DRV] Write Data: 56
[1956] [SCB] MATCH: Expected 56, Got 56
[1966] [DRV] Write Data: 56
[1966] [SCB] MATCH: Expected 56, Got 56
[1976] [DRV] Write Data: 56
[1976] [SCB] MATCH: Expected 56, Got 56
[1986] [DRV] Write Data: 56
[1986] [SCB] MATCH: Expected 56, Got 56
[1996] [DRV] Write Data: 56
[1996] [SCB] MATCH: Expected 56, Got 56
[2006] [DRV] Write Data: 56
[2016] [DRV] Write Data: 56
[2026] [DRV] Write Data: 56
[2036] [DRV] Write Data: 56
[2046] [DRV] Write Data: 56

========================================
          FIFO TRANSACTION LOG
========================================
 ID  |  Result  |  Data Value (Hex)
-----|----------|-------------------------
1    |  PASS    |  0x56
2    |  PASS    |  0x56
3    |  PASS    |  0x56
4    |  PASS    |  0x56
5    |  PASS    |  0x56
6    |  PASS    |  0x56
7    |  PASS    |  0x56
8    |  PASS    |  0x56
9    |  PASS    |  0x56
10   |  PASS    |  0x56
11   |  PASS    |  0x56
12   |  PASS    |  0x56
13   |  PASS    |  0x56
14   |  PASS    |  0x56
15   |  PASS    |  0x56
16   |  PASS    |  0x56
17   |  PASS    |  0x56
18   |  PASS    |  0x56
19   |  PASS    |  0x56
20   |  PASS    |  0x56
21   |  PASS    |  0x56
22   |  PASS    |  0x56
23   |  PASS    |  0x56
24   |  PASS    |  0x56
25   |  PASS    |  0x56
26   |  PASS    |  0x56
27   |  PASS    |  0x56
28   |  PASS    |  0x56
29   |  PASS    |  0x56
30   |  PASS    |  0x56
31   |  PASS    |  0x56
========================================
 TOTAL PASSED: 31
 TOTAL FAILED: 0
========================================

--- SIMULATION FINISHED AT 2156 ---
- top.sv:61: Verilog $finish
- S i m u l a t i o n   R e p o r t: Verilator 5.030 2024-10-27
- Verilator: $finish at 2ns; walltime 0.002 s; speed 1.110 us/s
- Verilator: cpu 0.002 s on 1 threads; alloced 41 MB
```

Figure 2: The Scoreboard output

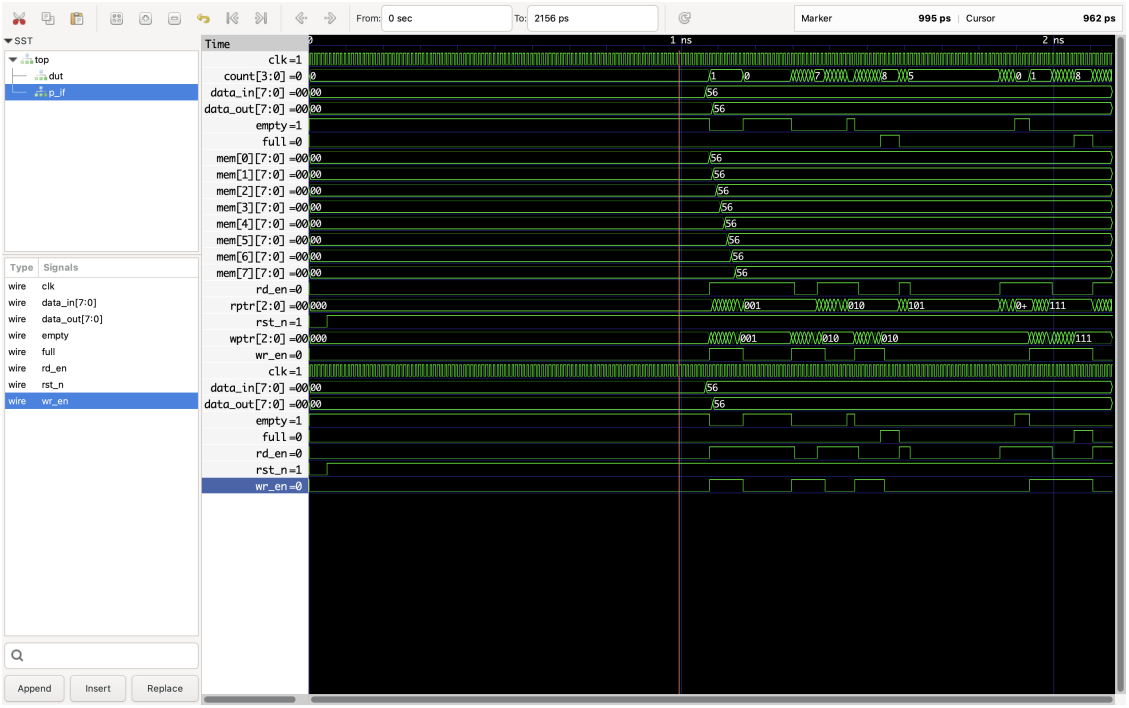

Figure 3: The Waveform output

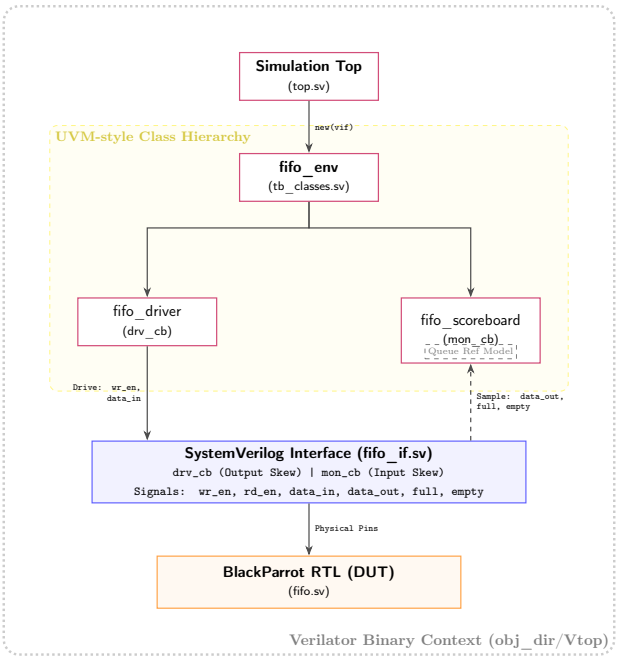

Figure 4: Final Verification Architecture: Integrated BlackParrot FIFO with SystemVerilog UVM-style Environment

3.2 Candidate Modules Chosen

Based on my prior work in visualizing the **BlackParrot 40-bit physical address routing hierarchy**,present in the official platform guide. I have selected modules that are central to this routing logic and the core execution pipeline. These choices allow me to transition from architectural visualization to functional verification:

- **BP-Stream Protocol Adapters:**
  - **Focus:** For the communications
  - **Role:** These act as translators that allow different parts of the chip to talk to each other using the 40-bit address routing logic.
  - **Verification Goal:** To ensure that data packets are sent and received correctly without getting "stuck" (deadlock) or lost during the handshake (Valid/Ready signals).
- **Integer Execution Unit (ALU):**
  - **Focus:** Integer arithmetic and logical operations.
  - **Role:** This is where the actual math (addition, subtraction, etc.) happens for the 64-bit RISC-V instructions.
  - **Verification Goal:** To confirm that every math operation produces the exact right answer by comparing the hardware's result against a perfect version of the ALU. For the ALU golden model, I plan to implement a behavioral SystemVerilog model that mirrors the RISC-V ISA specifications for 64-bit integer operations.
- **Register File (RegFile):**

- **Focus:** Register storage and access.
- **Role:** This is the internal storage where the processor keeps data it is currently working on.
- **Verification Goal:** To ensure that when the processor writes data to a specific "folder" (register), it can read that exact same data back later without any corruption.

4 Plan

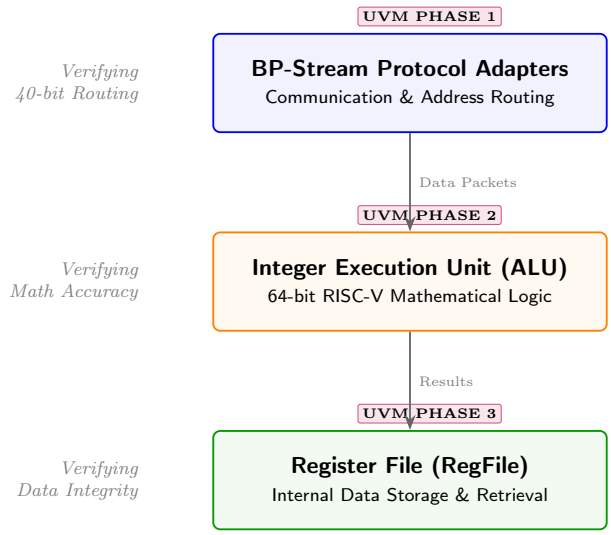

Figure 5: Project Roadmap: Sequential UVM Verification of Communication, Computation, and Storage Layers.

4.1 Verification Methodology: From Directed to UVM

The core of this project is to evolve the current verification suite from directed, script-based simulations to a scalable, UVM framework. My strategy involves three integrated testing tiers:

- **UVM-Wrapped Directed Tests (Baseline):** I will reuse existing high-value simulation scenarios (like *RISC-V ISA Simple Tests* and *Hello World*) by wrapping them into UVM-compatible sequences. This ensures the new testbench maintains backward compatibility with proven core boot flows.
- **Constrained Random Verification (CRV):** Using `uvm_sequence`, I will implement randomized stress tests inspired by the existing *Random Coherence Stress* examples. This approach will automatically generate edge-case instruction streams for the ALU and complex cache eviction scenarios for the BP-ME.
- **Stress and Corner-Case Testing (Targeted):** Specific "Hammer" tests will be converted into UVM tests to verify hardware stability under extreme conditions.

4.2 Risks In This Plan

I have identified two primary technical risks for this UVM implementation:

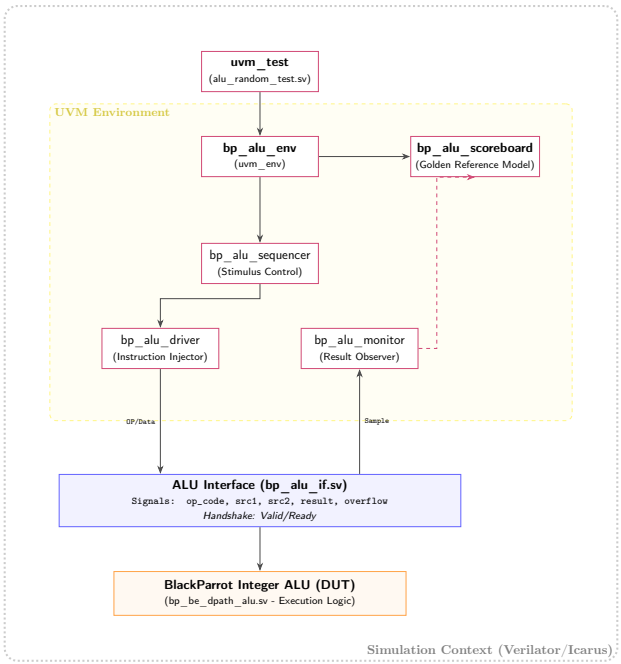

Figure 6: Proposed Architecture: UVM Environment for the BlackParrot Integer ALU

- **Verilator-UVM Feature Gaps:** Although Verilator has recently added UVM support, specific complex SystemVerilog constructs—such as advanced constrained randomization may still trigger tool-level limitations or bugs.
- **Computational Overhead:** Implementing a full-chip 40-bit address routing monitor in a cycle-accurate environment can be resource-intensive. This could potentially slow down simulation cycles, impacting the rapid iteration flow essential to BlackParrot development.

4.3 Backup Plan

- **Modular SV-Class Framework:** If Verilator's UVM implementation reaches a critical roadblock, I will pivot to a "UVM-lite" approach using pure SystemVerilog classes. This maintains Object-Oriented modularity and reusability while ensuring 100% compatibility with the current open-source toolchain.
- **Unit-Level Priority:** If full-hierarchy routing verification becomes computationally prohibitive, I will focus on high-fidelity unit-level verification for the individual **BP-Stream Adapters** and the **ALU**. This ensures core mathematical and protocol logic is fully verified regardless of system-level simulation speeds.

5 Timeline

- **Before May 25 (Community Bonding Period)**
  - Go through BlackParrot modules I selected (BP-Stream, ALU, Register File)
  - Understand existing simulation flow (bp_top, bp_me) and how testing is currently done
  - Set up and validate my environment with Verilator properly

  - Discuss with my mentor to finalize scope and expectations
  - Decide how I want to structure the UVM testbenches (agents, scoreboard, etc.)

- **May 25 – June 7 (Coding officially begins)**
  - Start with BP-Stream adapters (main priority)
  - Design the UVM structure and define transactions and interfaces
  - Implement basic components like driver and monitor
  - Run simple directed tests just to make sure everything works

- **June 8 – June 21**
  - Add constrained random sequences for BP-Stream
  - Implement scoreboard to check correctness of packet flow and handshake
  - Test edge cases like backpressure and possible deadlock scenarios
  - Debug and stabilize the environment

- **June 22 – July 5**
  - Start working on ALU verification
  - Build UVM components (driver, monitor, sequencer)
  - Create a reference model (golden ALU) to compare results
  - Test basic arithmetic operations first

- **July 6 – July 10 (Midterm Evaluation)**
  - Add constrained random testing for ALU
  - Cover edge cases like overflow and boundary values
  - Document work done so far
  - Submit working BP-Stream and partial ALU verification

- **July 11 – July 24**
  - Complete ALU verification with better coverage
  - Start Register File verification
  - Test read/write behavior and data consistency
  - Make sure all modules are working properly.

- **July 25 – August 10**
  - Integrate all testbenches together
  - Improve test coverage and clean up code
  - Handle any issues with Verilator and UVM support
  - Make things more reusable and modular.

- **August 11 – August 17 (Final Submission)**

  - Final documentation (how to use, what was implemented)
  - Clean up everything
  - Submit final PRs / deliverables

- **August 18 – August 31(Buffer)**
  - Extra time in case something breaks or takes longer
  - Improve coverage or add more test cases if time allows
  - Address mentor feedback

6 Deliverables

- **UVM Environment:** A complete, reusable UVM testbench architecture integrated with Verilator .
- **Verification IPs (VIPs):** Drivers, monitors, and sequencers for BlackParrot Stream, ALU, and Register File modules.
  - **BP-Stream:** Scoreboard and agents for handshake and packet verification.
  - **ALU:** Golden reference model and constrained random sequences for arithmetic/logic operations.
  - **Register File:** Comprehensive tests for read/write consistency and multi-port access.
- **Documentation:** A detailed technical report and `README` files explaining how to run simulations and interpret results.

7 Availability

- **Weekly Commitment:** I am committed to working full-time on this project (approximately 30–40 hours per week) to ensure all milestones are met within the 175-hour project scope.
- **Timezone:** IST (UTC +5:30) .
- **Prior Commitments:** No major academic conflicts during the GSoC period.

8 Schedule To Update Community

- **Daily Updates:** Short progress summaries and blocker discussions via GitHub(major) and the BlackParrot google group.
- **Weekly Syncs:** Detailed weekly reports shared with my mentor to review code progress and plan upcoming tasks.
- **Blog Posts:** Weekly technical blog posts documenting my journey, challenges (like Verilator UVM gaps), and milestones.

9 References

- BlackParrot repository
- Siemens UVM Cookbook